TransferIa · Documentos de cambios

Extraer caras, carpetas, tildes y rutas

Entrega 12 · Revisar caras antes del modelo, vista previa de carpeta, tildes y nombres largos

Sin commit: carpeta de trabajo al 27 de octubre de 2026

Comparado con: d6b94d6

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. Revisar las caras antes de crear el modelo 3
2.1 El problema 3
2.2 Cómo usarlo en la ventana 3
2.3 Cómo usarlo en la CLI 4
2.4 Probado: qué hay que borrar 5
2.5 De qué manera funciona 6
3. Vista previa de una carpeta 6
4. Tildes y ñ en las rutas 7
4.1 El problema 7
4.2 La solución 8
5. Nombres de video muy largos 8
6. Cómo se comprobó 9
Cómo probarlo tú 9
7. Lo que aprendimos 9
Glosario 10

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

> **Nota:** Estos cambios **todavía no tienen commit**: están en la carpeta de trabajo de `D:\TransferIa` al 27 de octubre. Se probaron con una copia de esos archivos y, para el "antes", con el commit `d6b94d6`. Todas las pruebas usan caras **generadas por IA**; los números marcados "(README)" son mediciones de la sesión de desarrollo escritas en el README que no se volvieron a medir.

Cambio	Interfaz	CLI	Qué gana el usuario
Revisar las caras antes de crear el modelo	Casilla en **Desde video** y ventana **Revisar caras extraídas**	`crear-modelo --extraer-caras`	Modelos más fieles: se borran las caras malas
Vista previa de una carpeta	**< Anterior**, **Siguiente >**, barra y flechas	—	Revisar el resultado foto por foto
Tildes y ñ en las rutas	—	—	Fotos como "maría josé.jpg" ya se leen y se guardan
Nombres largos	—	—	Videos bajados de internet ya no rompen la ruta

2. Revisar las caras antes de crear el modelo

2.1 El problema

Al crear un modelo desde un video (entrega 8), las caras se sacaban en memoria y se promediaban sin que se pudieran ver. Si el video tenía caras borrosas, tapadas, con gestos raros o de otra persona, entraban al promedio y el modelo se parecía menos. La versión anterior no tiene ninguna opción para verlas (primera captura de 2.3).

2.2 Cómo usarlo en la ventana

1. Pulsa **Desde video** y elige el video.
2. Marca **Revisar las caras antes de crear el modelo**. El botón pasa de **Crear modelo** a **Extraer caras**.
3. Pulsa **Extraer caras**. Se abre la carpeta `_en_proceso` y las caras van apareciendo a medida que se detectan.
4. Al terminar se abre la ventana **Revisar caras extraídas** y la carpeta de `persona_1` en el explorador.
5. Borra las caras malas en el explorador. Para otra persona, elígela y pulsa **Abrir carpeta**.
6. Pulsa **Crear modelo**: se crea con las caras que quedaron y queda cargado como fuente.

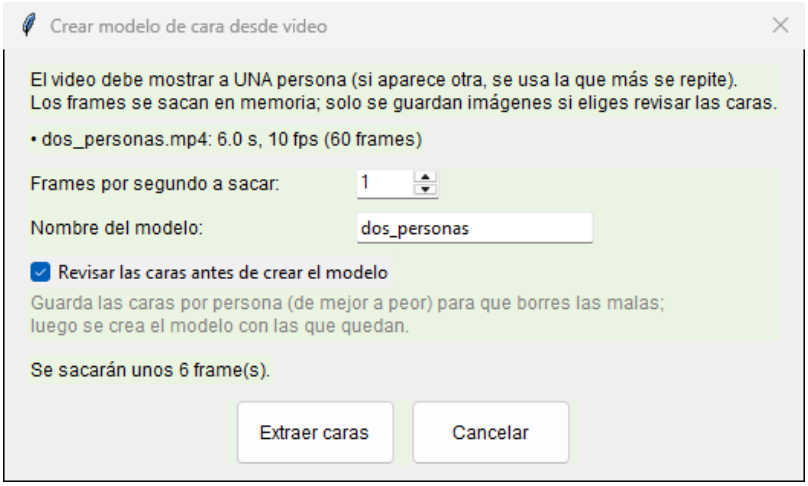


Figura 1. *Ventana "Desde video" con la casilla nueva marcada*

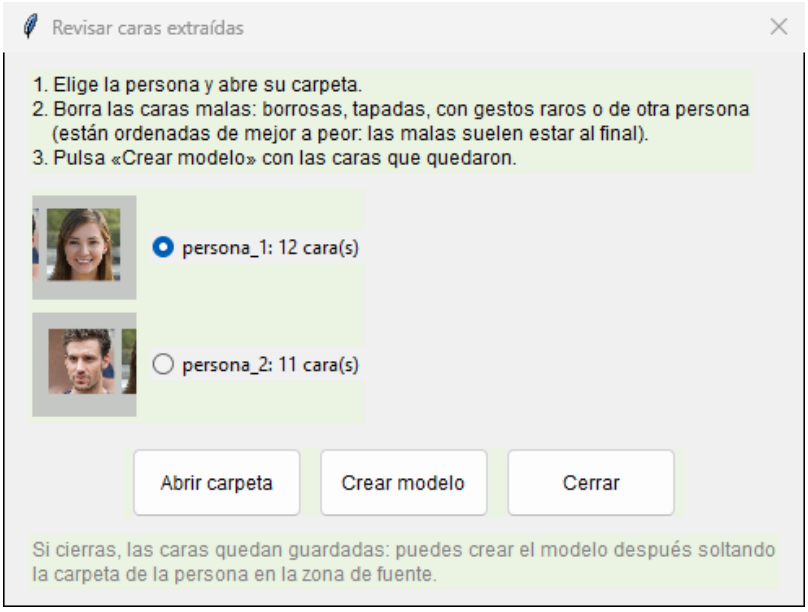


Figura 2. *Ventana "Revisar caras extraídas": una fila por persona con miniatura y cantidad*

> **Consejo:** Si cierras la ventana, las caras quedan guardadas. Después puedes soltar la carpeta de la persona en la zona de fuente para crear el modelo.

2.3 Cómo usarlo en la CLI

Son dos pasos: extraer y, después de revisar, crear el modelo con la carpeta.

```
> version anterior: python cli.py crear-modelo --video dos_personas.mp4 --fps 2 --extraer-caras
usage: cli.py crear-modelo [-h] [--fotos FOTOS [FOTOS ...]]
                           [--video VIDEO [VIDEO ...]] [--fps FPS]
                           --output OUTPUT [--overwrite]
cli.py crear-modelo: error: the following arguments are required: --output

> version nueva: python cli.py crear-modelo --video dos_personas.mp4 --fps 2 --extraer-caras
Las caras se van guardando en <prueba>\v\actual\src\caras_extraidas\dos_personas\_en_proceso
dos_personas.mp4: frame 1/~12 (0.0 s): 2 cara(s) util(es)
dos_personas.mp4: frame 2/~12 (0.5 s): 2 cara(s) util(es)
   ...
dos_personas.mp4: frame 12/~12 (5.5 s): 2 cara(s) util(es)
persona_1: 12 cara(s)
persona_2: 11 cara(s)
23 cara(s) guardadas en <prueba>\v\actual\src\caras_extraidas\dos_personas:
  persona_1: 12 cara(s)
  persona_2: 11 cara(s)
Revisa la carpeta, borra las caras malas (borrosas, tapadas, de otra persona) y crea el modelo:
  python cli.py crear-modelo --fotos "<prueba>\v\actual\src\caras_extraidas\dos_personas\persona_1" --ou
  tput src/modelos_caras/<nombre>.npy

> (otra vez, el mismo comando)
Error: Ya hay caras extraidas en <prueba>\v\actual\src\caras_extraidas\dos_personas; reemplazalas o elig
  e otro nombre
```

Figura 3. *La versión anterior no conoce la opción; la nueva extrae, y no pisa una extracción anterior sin --overwrite*

```
> tree src/caras_extraidas/dos_personas
src/caras_extraidas/dos_personas/
├── .transferia_caras       (marca: carpeta creada por TransferIa)
├── persona_1/   (12 caras)
│   ├── 001_dos_personas_0.0s.jpg
│   ├── 002_dos_personas_2.5s.jpg
│   ├── 003_dos_personas_0.5s.jpg
│   ├── ...
│   └── 012_dos_personas_3.0s.jpg
├── persona_2/   (11 caras)
│   ├── 001_dos_personas_0.0s.jpg
│   ├── 002_dos_personas_1.0s.jpg
│   ├── 003_dos_personas_0.5s.jpg
│   ├── ...
│   └── 011_dos_personas_2.5s.jpg
```

Figura 4. *Lo que queda en la carpeta: una subcarpeta por persona, de la mejor cara a la peor*

2.4 Probado: qué hay que borrar

Con el video de dos personas que se cruzan (6 s, sacando 2 frames por segundo) se extrajeron **23 caras**: 12 de una persona y 11 de la otra (en el segundo 3 se tapan y solo se ve una).

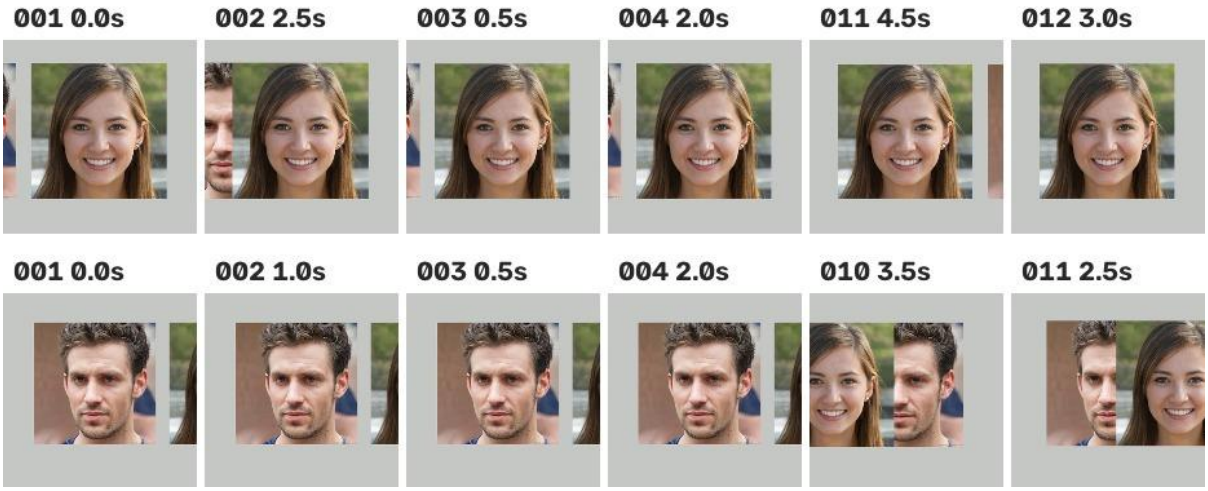


Figura 5. *Las 4 mejores y las 2 últimas de cada persona; el nombre dice el orden y el segundo del video*

Qué hay que mirar: en la fila de abajo, las dos últimas (010 y 011, de los segundos 3,5 y 2,5) son del momento en que se cruzan: **la otra persona aparece encima**. Son exactamente las que hay que borrar, y quedaron al final porque el orden es de mejor a peor.

```
> python cli.py crear-modelo --fotos src/caras_extraidas/dos_personas/persona_1 --output src/modelos_car
  as/persona_revisada.npy
Foto 1/12: 1 cara(s) util(es)
Foto 2/12: 2 cara(s) util(es)
   ...
Foto 12/12: 1 cara(s) util(es)
Cara descartada (002_dos_personas_2.5s.jpg): parece otra persona (similitud 0.06)
Cara descartada (010_dos_personas_3.5s.jpg): parece otra persona (similitud 0.04)
Aviso: las caras eran de mas de una persona; se uso la que mas se repite (12 de 14 caras)
Modelo creado con 12 cara(s) de 12 foto(s) -> <prueba>\v\actual\src\modelos_caras\persona_revisada.npy
Similitud promedio entre caras: 0.99 (buena)
Miniatura: <prueba>\v\actual\src\modelos_caras\persona_revisada.jpg
```

Figura 6. *Crear el modelo con la carpeta de persona_1*

Aun sin revisar, al crear el modelo con `persona_1` se descartaron solas las 2 caras de la otra persona que asomaban en el borde de dos recortes (similitud 0,06 y 0,04), y el modelo quedó con 12 caras y similitud 0,99.

2.5 De qué manera funciona

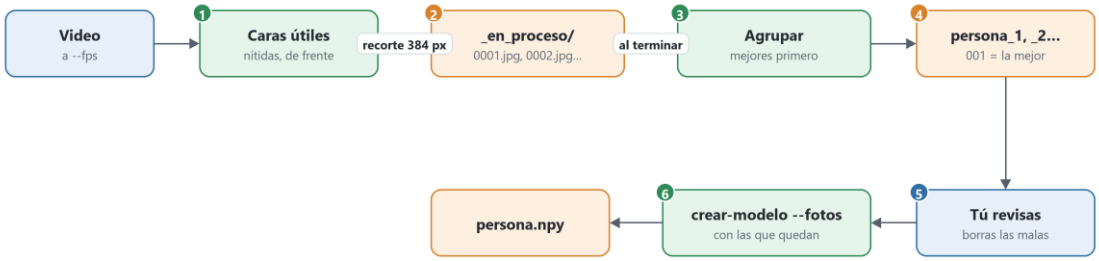


Figura 7. *De video a modelo pasando por la revisión*

1. Se recorre el video a los fps elegidos y se quedan las caras útiles (nítidas, de frente, de buen tamaño): las mismas reglas que al crear un modelo, en una función común (`_recolectar`).
2. Cada cara se guarda **apenas se detecta** en `_en_proceso/0001.jpg`, `0002.jpg`…: un recorte de 384×384 con la cara al centro ocupando la mitad. No se guarda en memoria la imagen entera.
3. Al terminar se agrupan por persona empezando por las mejores (así cada grupo arranca con una buena referencia). Dos caras del mismo frame nunca van al mismo grupo.
4. Los grupos se ordenan del más grande (`persona_1`) al más chico, y cada cara se mueve a su carpeta con un nombre que dice su orden y su segundo: `001_dos_personas_0.0s.jpg`.

¿Por qué no ordenar mientras se extrae? Para separar por persona y ordenar de mejor a peor hace falta haber visto **todas** las caras. Por eso se guardan primero en `_en_proceso` (así se ve el avance) y se reparten al final.

> **Importante:** Seguridad de los archivos: la carpeta lleva una marca (`.transferia_caras`). Con `--overwrite` solo se reemplaza una carpeta que tenga esa marca, y solo sus subcarpetas `persona_N` y `_en_proceso`. Una carpeta con otros archivos nunca se toca.

3. Vista previa de una carpeta

Con una carpeta de imágenes como destino (entrega 9), **Abrir Preview** mostraba solo la primera imagen. Para ver cómo quedaba otra había que procesar todo.

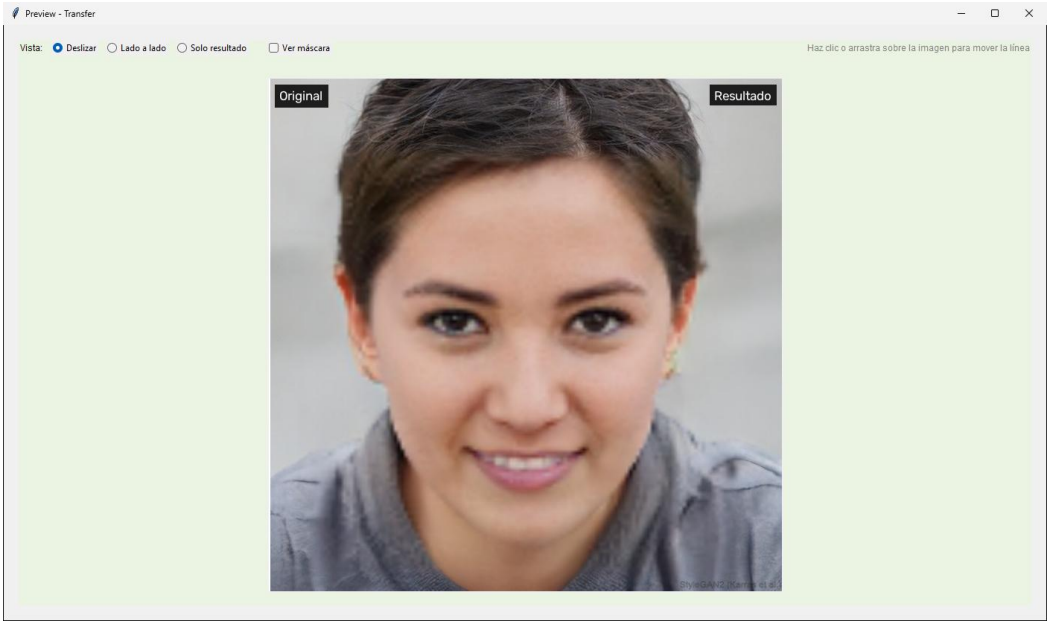


Figura 8. *Antes: solo la primera imagen, sin forma de pasar a otra*

Ahora abajo hay una barra (como la de los videos), los botones **< Anterior** y **Siguiente >** y las flechas ← → del teclado. Al lado se ve la posición y el nombre de la imagen.

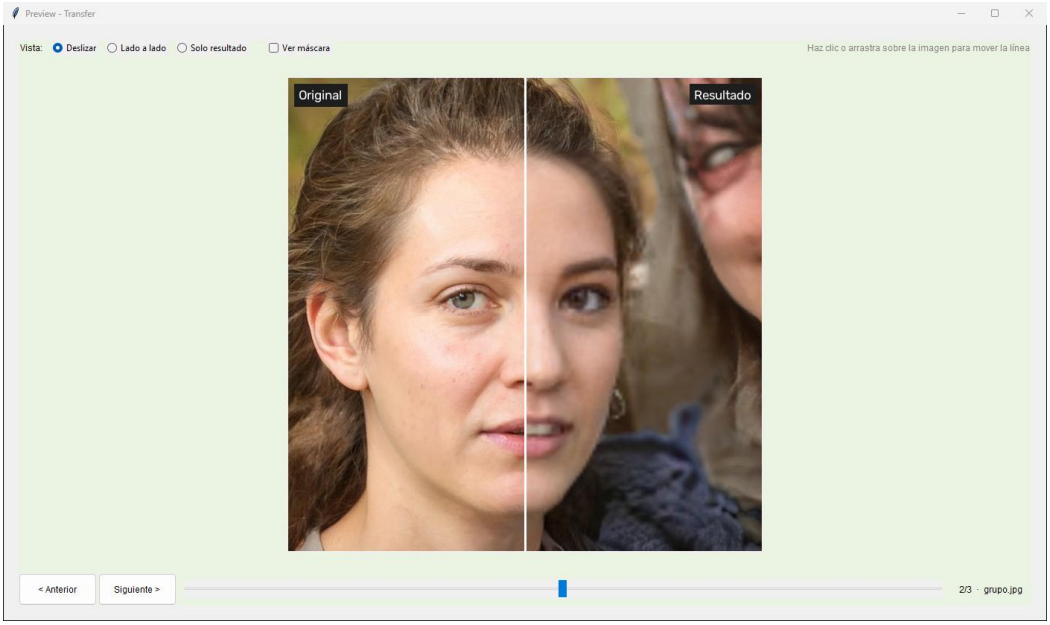


Figura 9. *Ahora: imagen 2 de 3 (grupo.jpg) después de pulsar "Siguiente >"*

Cada imagen se procesa al llegar a ella, en segundo plano, con las opciones actuales.

4. Tildes y ñ en las rutas

4.1 El problema

En Windows, `cv2.imread` y `cv2.imwrite` de OpenCV **no aceptan rutas con tildes o ñ**. `imread` devuelve "nada" y `imwrite` es peor: **dice que guardó, pero no crea el archivo**.

```
> version anterior: python cli.py process --source fuente.jpg --target "Fotos de la señora Muñoz/maría j
  osé.jpg" --output salida/e12_tilde_antes.jpg
[ WARN:0@8.587] global loadsave.cpp:278 cv::findDecoder imread_('<prueba>\datos\Fotos de la señora Muñoz
  \maría josé.jpg'): can't open/read file: check file path/integrity
Error: No se pudo leer la imagen destino: <prueba>\datos\Fotos de la señora Muñoz\maría josé.jpg

> version anterior: python cli.py process --source fuente.jpg --target destino.jpg --output "salida/seño
  ra.jpg"
Resultado guardado en: <prueba>\salida\señora.jpg

> dir /b salida | find /c "señora"   (¿existe el archivo?)
0

> version nueva: python cli.py process --source fuente.jpg --target "Fotos de la señora Muñoz/maría josé
  .jpg" --output "salida/señora.jpg"
Resultado guardado en: <prueba>\salida\señora.jpg

> dir /b salida | find "señora"
señora.jpg
```

Figura 10. *Antes no lee "maría josé.jpg" y dice haber guardado "señora.jpg" sin crearlo; ahora ambas funcionan*

Qué hay que mirar: en la versión anterior, el segundo comando dice "Resultado guardado en: …señora.jpg", pero la búsqueda en la carpeta da **0** archivos. Es el peor tipo de error: el programa dice que todo salió bien.

4.2 La solución

Un módulo nuevo, `core/Imagenes.py`, con dos funciones que reemplazan a `cv2.imread` y `cv2.imwrite` en todo el proyecto (6 archivos). Leen y escriben los **bytes** con numpy (que sí acepta esas rutas) y solo usan OpenCV para decodificar o codificar la imagen en memoria:

```
def leer(ruta, flags=cv2.IMREAD_COLOR):
    try:
        datos = np.fromfile(ruta, dtype=np.uint8)      # bytes del archivo: acepta tildes y ñ
    except OSError:
        return None
    return cv2.imdecode(datos, flags) if datos.size else None


def guardar(ruta, imagen, parametros=()):
    ok, datos = cv2.imencode(os.path.splitext(ruta)[1] or ".png", imagen, list(parametros))
    if not ok:
        return False
    datos.tofile(ruta)                                  # y si falla, devuelve False de verdad
    return True
```

El resultado es el mismo que con OpenCV (el formato sale de la extensión, y `imdecode` también aplica la orientación EXIF de las fotos de celular).

5. Nombres de video muy largos

Windows no acepta rutas de más de **260 caracteres** (salvo que se activen las rutas largas, que en este PC están desactivadas). Los videos bajados de internet suelen tener nombres enormes, y al extraer caras el nombre aparece **dos veces**: en la carpeta y en cada archivo.

Por eso el nombre sugerido y la carpeta por defecto se acortan a **40 caracteres** (`LARGO_NOMBRE`), y el del video dentro del nombre de cada cara a **30** (`LARGO_VIDEO_EN_CARA`).

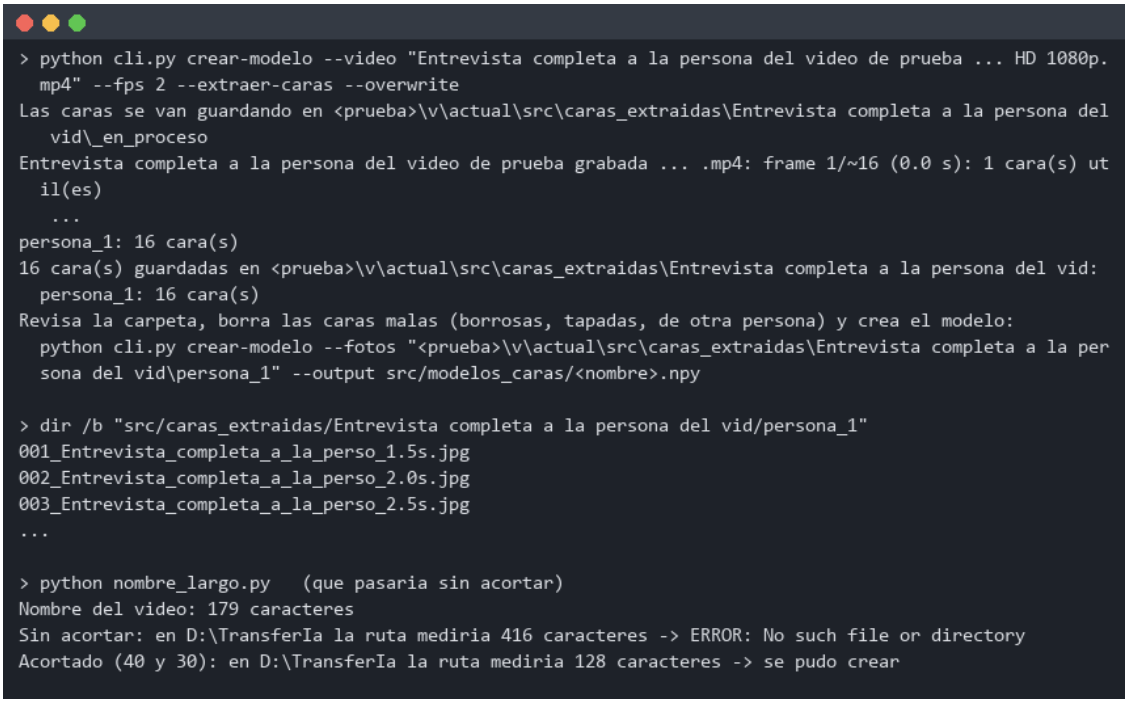


Figura 11. *Un video de 179 caracteres: carpeta y archivos acortados; y la prueba de qué pasaría sin acortar*

Sin acortar, la ruta de una cara mediría **416 caracteres** y Windows no la crea. Acortada, **128**.

> **Nota:** Las pruebas se hicieron en una unidad virtual `T:` (con `subst`) para tener una ruta tan corta como `D:\TransferIa`; las longitudes mostradas son las que tendría en `D:\TransferIa`.

6. Cómo se comprobó

- Extracción de un video de dos personas con la versión anterior (no existe la opción) y la nueva; segundo intento sin --overwrite; modelo creado con la carpeta.
- En la ventana: "Desde video" con la casilla, "Revisar caras extraídas" y la vista previa de una carpeta antes y después.
- Una foto en "Fotos de la señora Muñoz/maría josé.jpg" y una salida "señora.jpg", con las dos versiones.
- Un video de 179 caracteres de nombre; creación real de la ruta con y sin acortar.

Cómo probarlo tú

1. En **Desde video**, marca **Revisar las caras antes de crear el modelo** y extrae las caras de un video tuyo.
2. Borra en el explorador las que tengan a otra persona o estén borrosas (suelen estar al final).
3. Pulsa **Crear modelo** y compara con un modelo creado sin revisar.

7. Lo que aprendimos

- Dejar que la persona revise lo que la máquina eligió (y ordenarlo de mejor a peor) hace fácil la revisión.
- Guardar el avance apenas se tiene (cara por cara) muestra el progreso y no llena la memoria.
- Un "guardé bien" que no es verdad es peor que un error: hay que comprobar lo que dicen las funciones.
- Los nombres de archivo que vienen de afuera hay que limpiarlos y acortarlos.

- Nunca borrar una carpeta que el programa no creó: marcarla al crearla.

Glosario

Término	Significado
Recorte de revisión	Imagen cuadrada de 384 px con una cara al centro, para mirarla y crear el modelo.
Ruta larga	Ruta de más de 260 caracteres; Windows no la acepta si no se activa una opción del sistema.
EXIF	Datos que guarda la cámara dentro de la foto, entre ellos cómo estaba girado el celular.
imdecode / imencode	Funciones de OpenCV que convierten bytes en imagen y al revés, sin tocar archivos.